

```
$ spring run DomainLab --defense=0..100
```

Spring Boot × DDD

// 從貧血模型到完整領域防線

0%

貧血模型

25%

Value Object

50%

充血實體

75%

Aggregate

100%

Event / BC

\$ cat defense-plan.md

今日路線：一個訂單模型的五道防線

0% / 病灶盤點

魔法字串、規則散落、非法狀態

25% / 值物件

非法值建不出來

50% / 充血實體

非法狀態不可能存在

75% → 100% / 邊界

Aggregate、Event、Bounded Context

目標：每往右一格，就少一類「資料怎麼會變成這樣」的靈異事件。

PART 01 — 0%

貧血模型

Entity 只是資料袋，規則沒有家

Transaction Script 的日常

```
1  @Entity
2  public class Order {    // 只有資料, 沒有行為
3      private String status; // 魔法字串
4      private double total;  // 精度看運氣
5      // getter / setter 全開
6  }
7
8  @Service
9  public class OrderService { // 規則全塞在這
10     public void pay(Long id, double amount) {
11         Order order = repo.findById(id).get();
12         if (!"PENDING".equals(order.getStatus()))
13             throw new IllegalStateException();
14         order.setStatus("PAID");
15         order.setPaidAmount(amount); // 負數也照收
16         emailService.send(order);
17         repo.save(order);
18     }
19 }
```

SMELL REPORT

- × 「何時能付款」的規則寫在 Service
- × 同一條規則在 12 個地方複製漂移
- × `setStatus("SHIPPED")` 誰都能呼叫
- × PM 說「付款」, 程式寫 `setStatus`

規則沒有家 → 非法狀態隨時出現

PART 02 — 25%

Value Object

非法值，從此建不出來

原始型別 → 值物件

```
1 @Entity
2 public class Order {
3     @Enumerated(EnumType.STRING)
4     private OrderStatus status; // 打錯直接編譯不過
5
6     @Embedded
7     private Money total;        // 負數金額不存在
8 }
```

```
1 public record Money(BigDecimal amount, Currency currency) {
2     public Money {
3         if (amount.signum() < 0)
4             throw new IllegalArgumentException("金額不可為負");
5     }
6 }
```

驗證只寫一次、寫在型別裡 — 不是「每次記得檢查」，是「非法值不存在」。

PART 03 — 50%

充血模型與領域語言

行為搬回實體，程式碼說業務的話

實體自我守護

```
1  @Entity
2  public class Order {
3      @Enumerated(EnumType.STRING)
4      private OrderStatus status;
5      @Embedded private Money total;
6
7      protected Order() {} // 只給 JPA 用
8
9      public void pay(Money amount) {
10         if (status != OrderStatus.PENDING)
11             throw new OrderNotPayableException(id);
12         if (!amount.equals(total))
13             throw new PaymentMismatchException();
14         this.status = OrderStatus.PAID;
15     }
16
17     // 沒有 setStatus()
18 }
```

Service 變薄

```
1  @Service
2  public class OrderService {
3      @Transactional
4      public void pay(Long id, Money amount) {
5          Order order = findOrder(id);
6          order.pay(amount);
7          // 找到人，請他做事
8      }
9  }
```

Ubiquitous Language

PM 說「訂單付款」，程式碼寫 `order.pay()` —— 翻譯損耗歸零

狀態轉移必經守衛 → 非法狀態不可能存在

測試也跟著解放

```
1  @Test
2  void 已出貨的訂單不能取消() {
3      Order order = shippedOrder(); // 純物件，不用 @SpringBootTest
4
5      assertThatThrownBy(order::cancel)
6          .isInstanceOf(OrderAlreadyShippedException.class);
7  }
```

規則只有一份

住在它所屬的物件裡，改一處全站生效

測規則不碰 DB

領域邏輯 = 純物件行為，毫秒級驗證

動手拉拉看

```
$ spring run DomainLab
```

● 危險

0% 貧血模型



0%

25%

50%

75%

100%

- x 金額用 `double`、狀態用魔法字串
- x 商業規則散落 `N` 個 `Service`，改不齊
- x `setter` 全開，狀態隨人改壞
- x 想改哪張表就改哪張，一致性看緣分
- x 跨模組直接互呼，改一處動全身

PART 04 — 75%

Aggregate 與 Repository

一致性，終於有了邊界

聚合根是守門人

```
1  @Entity
2  public class Order { // Aggregate Root
3      @OneToMany(cascade = ALL, orphanRemoval = true)
4      private List<OrderItem> items = new ArrayList<>();
5
6      public void addItem(Product product, int qty) {
7          if (status != OrderStatus.PENDING)
8              throw new OrderLockedException(id);
9          items.add(OrderItem.of(product, qty));
10         this.total = calculateTotal(); // 不變條件
11     }
12
13     public List<OrderItem> items() {
14         return List.copyOf(items); // 唯讀副本
15     }
16 }
```

AGGREGATE RULES

1. 想動邊界內的東西 → 跟 root 說
2. 一個聚合 = 一個交易
(@Transactional 對齊)
3. 跨聚合只用 ID 參照，不抱整個物件圖

Repository 只服務聚合根：
沒有 OrderItemRepository

PART 05 — 100%

Domain Event 與 Bounded Context

模組之間，只透過事件說話

宣告「發生了什麼」

```
1  @Entity
2  public class Order
3      extends AbstractAggregateRoot<Order> {
4
5      public void pay(Money amount) {
6          ensurePayable(amount);
7          this.status = OrderStatus.PAID;
8          registerEvent(new OrderPaid(id, total));
9      }
10 }
```

save() 時自動發布

@TransactionalEventListener 保證 commit 成功後才處理

別的模組自己決定要不要關心

```
1  @Component
2  class ShippingHandler {
3      @TransactionalEventListener
4      void on(OrderPaid event) {
5          shippingService.prepare(event.orderId());
6      }
7  }
8
9  @Component
10 class NotificationHandler {
11     @TransactionalEventListener
12     void on(OrderPaid event) {
13         emailService.sendReceipt(event.orderId());
14     } // 訂單模組完全不認識它
15 }
```

Bounded Context : 同名不同模型



同一個「Product」在不同上下文是不同的模型 — 這不是重複，是解耦。不再用 50 個欄位的萬能 Entity 撐全站。

0% vs 100%

指標	0%	100%
非法值	負數金額、錯字狀態照收	Value Object : 建構即驗證
非法狀態	setter 全開，隨人改	狀態轉移必經守衛
規則位置	散落 N 個 Service	唯一的家，在所屬物件裡
一致性	看緣分	Aggregate 交易邊界內恆成立
業務語意	setStatus("PAID") 要人腦翻譯	order.pay() 與 PM 同語言
模組耦合	改一行，全站重測	Event + Bounded Context 分家

常見疑慮

JPA Entity 當領域模型，會被綁死嗎？

務實折衷可接受：protected 建構子給 JPA、@Embedded 放值物件，領域行為照寫。完全隔離是 100% 之後的選修題。

跟 @Valid 是什麼關係？

@Valid 守輸入格式（DTO），值物件與實體守商業規則。前者擋門口，後者守核心，兩層都要有。

小專案也要 100%？

每一格都是穩定停靠點：CRUD 後台停在 25% 就划算；有真實商業規則的核心模組至少到 50%。

今日重點

25%

值物件：非法值建不出來

50%

充血實體：非法狀態不可能存在

75%

Aggregate：一致性有邊界

100%

Event / BC：模組只透過事件說話

完美不是「畫滿架構圖」，是每條規則有唯一的家。

Defense complete.

下一步：互動版文章 `/springboot/ddd-in-spring-boot`

```
$ spring run DomainLab --defense=100
```

```
value object → entity → aggregate → event
```